CENTRO UNIVERSITÁRIO CEUNI - FAMETRO
Curso de Sistemas de Informação | Disciplina: Tecnologia Web I
Professor: Alexsander Farias | 2026.1 - Manaus, AM

AULA PRÁTICA 04 - PASSO A PASSO

CSS3 Parte 2 + Flexbox: Posicionamento e Layout Responsivo

OK 655 + OK 656 | Projeto: Portfólio Pessoal Estático

IDENTIFICAÇÃO DA AULA

Objetos de Conhecimento	OK 655 - Posicionamento e Display | OK 656 - Layouts com Flexbox
Duração	100 minutos
Pré-requisito	Aula 03 concluída - style.css com reset, tipografia e Box Model
Ferramentas	VS Code + Live Server + Chrome DevTools (F12)
Entregável	Portfólio com layout completo e responsivo: header flexbox, colunas, grid de cards e menu sticky

OBJETIVO DA AULA

Aplicar posicionamento CSS (display, position) e construir o layout completo do portfólio utilizando Flexbox. Ao final, o portfólio terá um cabeçalho com menu alinhado, seção Sobre Mim em duas colunas, grid de cards de projetos e responsividade básica para dispositivos móveis.

PASSO 1 - DISPLAY: COMO OS ELEMENTOS SE COMPORTAM

A propriedade display define como um elemento ocupa espaço na página. É a base de todos os layouts CSS.

Tag / Propriedade	Descrição / Uso
`display: block`	Elemento ocupa a linha inteira. Elementos de bloco padrão: , <p>, <h1-h6>, <section>.
`display: inline`	Elemento ocupa apenas o espaço do seu conteúdo. Padrão: , <a>, .
`display: inline-block`	Mistura dos dois: fica na mesma linha, mas aceita width e height.
`display: none`	Remove o elemento visualmente E do fluxo do documento (invisível e sem ocupar espaço).
`display: flex`	Ativa o Flexbox no elemento. Seus filhos diretos se tornam flex items. (Passo 3)

```
/* Exemplo prático: botões lado a lado */
.botao {
  display: inline-block;
  padding: 0.5rem 1.5rem;
  background-color: var(--cor-primaria);
```

```
  color: white;
  border-radius: 4px;
  margin-right: 0.5rem;}
```

PASSO 2 - POSITION: POSICIONANDO ELEMENTOS

A propriedade position controla como um elemento é posicionado em relação ao fluxo normal da página ou a elementos pai.

Tag / Propriedade	Descrição / Uso
`position: static`	Padrão — o elemento segue o fluxo normal do documento.
`position: relative`	Posiciona relativo a onde o elemento naturalmente estaria. Não sai do fluxo.
`position: absolute`	Posiciona em relação ao ancestral mais próximo com position não-static. Sai do fluxo.
`position: fixed`	Fixo em relação à viewport. Não se move ao rolar a página.
`position: sticky`	Híbrido: se comporta como relative até atingir um limite (top/left), então fica fixo.
`top / right / bottom / left`	Usadas com position para definir a posição final do elemento.
`z-index`	Controla a sobreposição (qual elemento fica na frente). Valores maiores ficam na frente.

2.1 Menu de navegação fixo com position: sticky

Adicione ao style.css para que o header fixe no topo ao rolar a página:

```
/* Menu sticky — fica fixo no topo ao rolar */
header {
  position: sticky;
  top: 0;                        /* trava no topo */
  z-index: 1000;                 /* fica na frente de outros elementos */
  background-color: var(--cor-secundaria);
  /* ... outros estilos já definidos na Aula 03 */}
```

> **sticky vs fixed**
> position: fixed remove o elemento do fluxo e pode sobrepor conteúdo. position: sticky é mais inteligente: ocupa seu espaço normalmente e só trava quando a rolagem atinge o limite definido.

PASSO 3 - FLEXBOX: LAYOUTS FLEXÍVEIS

Flexbox (Flexible Box Layout) é um modelo de layout CSS que facilita o alinhamento e a distribuição de elementos em uma linha ou coluna. É ativado com display: flex no elemento pai (flex container).

Vocabulário do Flexbox:

Termo	O que é
Flex Container	O elemento com display: flex. Controla como os filhos se organizam.
Flex Items	Os filhos diretos do flex container.
Eixo Principal	A direção padrão do flex — horizontal (row) ou vertical (column).
Eixo Cruzado	Perpendicular ao eixo principal.

Propriedades do Flex Container (pai):

Tag / Propriedade	Descrição / Uso
`flex-direction`	Define a direção dos itens: row (padrão, horizontal), column (vertical), row-reverse, column-reverse.
`justify-content`	Alinha os itens no eixo principal: flex-start, flex-end, center, space-between, space-around, space-evenly.
`align-items`	Alinha os itens no eixo cruzado: flex-start, flex-end, center, stretch (padrão).
`flex-wrap`	Define se os itens quebram linha: nowrap (padrão), wrap, wrap-reverse.
`gap`	Espaçamento entre os flex items. Substitui margin entre itens.

Propriedades dos Flex Items (filhos):

Tag / Propriedade	Descrição / Uso
`flex`	Atalho para flex-grow, flex-shrink e flex-basis. Use flex: 1 para que o item cresça e ocupe o espaço disponível.
`align-self`	Sobrescreve o align-items só para este item específico.
`order`	Reordena os itens sem mudar o HTML. Padrão é 0.

3.1 Flexbox no Header: logo + menu alinhados

Adicione ao style.css:

```
/* ====================================================
   HEADER COM FLEXBOX
   ==================================================== */
header {
  display: flex;
  justify-content: space-between; /* logo à esquerda, nav à direita */
  align-items: center;            /* alinhamento vertical centralizado */
  padding: 1rem 2rem;
  position: sticky;
  top: 0;
  z-index: 1000;
  background-color: var(--cor-secundaria);
}

.header-container {
  display: flex;
  align-items: center;
  gap: 1rem;
}
```

```
nav ul {
  display: flex;
  list-style: none;
  gap: 0.5rem;}
```

3.2 Seção Sobre Mim em duas colunas

Precisamos envolver o conteúdo em um div.sobre-container no HTML. Adicione no index.html:

```
  <section id="sobre">
    <h2>Sobre Mim</h2>
    <div class="sobre-container">
      <img src="img/foto-perfil.jpg" alt="Foto de [Seu Nome]" class="foto-
sobre">
      <div class="sobre-texto">
        <p>Olá! Sou <strong>Seu Nome</strong>...</p>
        <!-- demais parágrafos e listas -->
      </div>
    </div>  </section>
```

Agora o CSS para o layout de duas colunas:

```
/* ====================================================
   SEÇÃO SOBRE MIM — duas colunas com Flexbox
   ==================================================== */
.sobre-container {
  display: flex;
  align-items: flex-start;
  gap: 3rem;
}

.foto-sobre {
  width: 200px;
  height: 200px;
  border-radius: 50%;
  object-fit: cover;
  border: 4px solid var(--cor-primaria);
  flex-shrink: 0;                /* impede a foto de encolher */
}

.sobre-texto {
  flex: 1;                       /* ocupa o espaço restante */}
```

3.3 Grid de cards de projetos com flex-wrap

```
/* ====================================================
   GRID DE PROJETOS — Flexbox com wrap
   ==================================================== */
#projetos {
  background-color: var(--cor-fundo);
}

.projetos-grid {
  display: flex;
  flex-wrap: wrap;               /* quebra linha quando não cabe */
  gap: 1.5rem;
  justify-content: flex-start;
```

```
}

.card-projeto {
  flex: 0 0 280px;               /* não cresce, não encolhe, base 280px */
  background-color: white;
  border-radius: 8px;
  border: 1px solid #DDDDDD;
  padding: 1.5rem;
  box-shadow: 0 2px 8px rgba(0,0,0,0.08);
  transition: transform 0.2s, box-shadow 0.2s;
}

.card-projeto:hover {
  transform: translateY(-4px);   /* sobe levemente ao passar o mouse */
  box-shadow: 0 6px 20px rgba(0,0,0,0.15);}
```

Não esqueça de adicionar a classe projetos-grid no HTML:

```
  <section id="projetos">
    <h2>Projetos</h2>
    <div class="projetos-grid">
      <article class="card-projeto">...</article>
      <article class="card-projeto">...</article>
    </div>  </section>
```

PASSO 4 - RESPONSIVIDADE COM MEDIA QUERIES

Media queries permitem aplicar estilos diferentes dependendo do tamanho da tela. Use para adaptar o layout a dispositivos móveis.

```
/* ====================================================
   RESPONSIVIDADE — Mobile First
   max-width: 768px = telas menores que tablet
   ==================================================== */
@media (max-width: 768px) {

  /* Header empilha verticalmente */
  header {
    flex-direction: column;
    align-items: flex-start;
    gap: 1rem;
  }

  /* Menu em coluna */
  nav ul {
    flex-direction: column;
    gap: 0.25rem;
  }

  /* Sobre Mim: foto acima do texto */
  .sobre-container {
    flex-direction: column;
    align-items: center;
  }

  /* Cards ocupam largura total */
  .card-projeto {
    flex: 1 1 100%;
  }

  /* Títulos menores */
```

```
  h1 { font-size: 1.8rem; }
  h2 { font-size: 1.4rem; }}
```

Testando no mobile com DevTools

Pressione F12 > ícone de celular/tablet (Toggle Device Toolbar) > escolha um dispositivo (ex: iPhone 12) para simular a visualização mobile sem precisar de um celular real.

PASSO 5 - TOQUES FINAIS E REVISÃO GERAL

1 — Seção Formação estilizada

Adicione margin e padding na seção de formação. Use listas com list-style-type personalizado se desejar.

2 — Botão de envio com transição

Adicione transition: background-color 0.2s ao button[type='submit'] para suavizar o efeito de hover.

3 — Scroll suave

Adicione html { scroll-behavior: smooth; } para que os links âncora rolem suavemente.

4 — Teste em diferentes tamanhos

Use o DevTools para testar em 375px (mobile), 768px (tablet) e 1200px (desktop).

```
/* Bônus: scroll suave e seleção de texto com a cor da marca */
html {
  scroll-behavior: smooth;
}

::selection {
  background-color: var(--cor-primaria);
  color: white;}
```

ERROS COMUNS NESTA AULA

Erro comum	Como corrigir
Flexbox não alinha como esperado	Confirme que display: flex está no elemento PAI (container), não nos filhos (items).
justify-content não funciona	justify-content alinha no eixo principal (horizontal por padrão). Se os itens já ocupam todo o espaço, não há espaço para distribuir.
Foto não ficou ao lado do texto	Verifique se o .sobre-container tem display: flex e se tanto a foto quanto o .sobre-texto são filhos diretos desse container.
Cards quebram mas ficam desalinhados	Adicione align-items: flex-start ao .projetos-grid para cards de alturas diferentes ficarem alinhados pelo topo.

Media query não aplica	As media queries devem vir depois dos estilos base no arquivo (em cascata). Verifique a sintaxe: @media (max-width: 768px) { }

CHECKLIST DO ENTREGÁVEL - AULA 04

☐	header com display: flex, justify-content: space-between e align-items: center
☐	Header com position: sticky e z-index definido
☐	Nav ul com display: flex e gap para links em linha
☐	.sobre-container com display: flex e foto + texto em duas colunas
☐	Foto de perfil com flex-shrink: 0 e .sobre-texto com flex: 1
☐	.projetos-grid com display: flex, flex-wrap: wrap e gap
☐	.card-projeto com efeito :hover (transform + box-shadow)
☐	html { scroll-behavior: smooth } aplicado
☐	@media (max-width: 768px) com layout empilhado (flex-direction: column)
☐	Portfólio testado e funcional em viewport mobile (375px) via DevTools

Próxima Aula - Aula 05: Git, GitHub e GitHub Pages

Na última aula você vai versionar o projeto com Git, criar um repositório público no GitHub e publicar o portfólio online com GitHub Pages - saindo com um link público real do seu portfólio na internet!

gov.br Documento assinado digitalmente
ALEXSAND FARIAS DE SOUZA
Data: 04/03/2026 17:09:20-0300
Verifique em https://validar.iti.gov.br

Prof. Alexsander Farias
Tecnologia Web I - CEUNI FAMETRO

Aluno(a)
Matrícula: _____

Manaus, AM - 2026.1